REFERENCE MANUAL

Git Command Cheat Sheet

Complete guide for managing local repositories, remote sync, branching, and historical operations.

1. CREATE NEW LOCAL REPO & MOVE TO REMOTE

Step-by-Step Workflow (From Scratch to Remote)

```
# 1. Initialize local repository
git init

# 2. Add all existing files to staging area
git add .

# 3. Make initial commit
git commit -m "Initial commit"

# 4. Rename default branch to 'main'
git branch -M main

# --- OPTION A: If Remote Repo does NOT exist yet (Create via GitHub CLI) ---
gh repo create <repo-name> --public --source=. --remote=origin --push

# --- OPTION B: If Remote Repo was created manually via Web UI (GitHub/GitLab) ---
git remote add origin <https://github.com/username/repo-name.git>
git push -u origin main
```

2. LOCAL REPOSITORY SETUP & CLONES

COMMAND	DESCRIPTION
`git init`	Initialize a new local Git repository in the current directory.
`git init <project-name>`	Create a new directory and initialize it as a Git repository.
`git clone <remote-url>`	Copy an existing repository from a remote host to your local machine.

3. REMOTE MANAGEMENT

COMMAND	DESCRIPTION
`git remote -v`	List all configured remote repositories along with their fetch/push URLs.
`git remote add <name> <url>`	Connect your local repo to a new remote repository location.
`git remote set-url <name> <new-url>`	Change the URL associated with an existing remote alias.
`git remote remove <name>`	Remove a remote server link from your local configuration.

4. STAGING & FILE TRACKING

COMMAND	DESCRIPTION
`git status`	View working directory status, untracked files, and staged changes.
`git add <file>`	Add a specific file to the staging area for the next commit.
`git add .`	Stage all modified, new, and deleted files in the working directory.
`git add -p`	Interactively review and select specific code chunks (hunks) to stage.
`git rm <file>`	Remove a file from both working directory and staging area.
`git rm --cached <file>`	Unstage a file while keeping the actual file in your local directory.
`git mv <old-path> <new-path>`	Move or rename a file, folder, or link, staging the change automatically.

5. COMMITTING CHANGES

COMMAND	DESCRIPTION
`git commit -m "Message"`	Save staged snapshot with a clear commit message.
`git commit -am "Message"`	Automatically stage tracked, modified files and commit in one step.
`git commit --amend -m "Msg"`	Modify the most recent commit message or include newly staged changes.

6. BRANCHING & MERGING

COMMAND	DESCRIPTION
`git branch`	List local branches (* marks active working branch).
`git branch -a`	List all local and remote-tracking branches.
`git branch <branch-name>`	Create a new branch at current HEAD position.
`git switch <branch-name>`	Switch to an existing branch (modern alternative to checkout).
`git checkout -b <branch>` `git switch -c <branch>`	Create a new branch and switch to it immediately.
`git merge <branch-name>`	Combine history from specified branch into active working branch.
`git branch -d <branch-name>`	Delete a local branch safely (checks if changes are merged).
`git branch -D <branch-name>`	Force delete a local branch regardless of merge status.
`git push origin --delete <branch>`	Delete a branch from the remote host repository.

7. SYNCING & UPDATING

COMMAND	DESCRIPTION
`git fetch origin`	Download remote objects & refs without altering working directory files.

COMMAND	DESCRIPTION
`git pull origin <branch>`	Fetch remote changes and immediately merge into active branch.
`git pull --rebase origin <branch>`	Fetch remote changes and rebase current branch commits on top.
`git push origin <branch>`	Upload local commits to the specified remote branch.
`git push -u origin <branch>`	Push and configure tracking link so future git push/pull omit arguments.
`git push --force-with-lease`	Safely force-push without overwriting others' unexpected remote work.

8. INSPECTING & HISTORY

COMMAND	DESCRIPTION
`git log`	Display full commit history with SHA-1 hashes, author, date, and messages.
`git log --oneline`	Display condensed single-line history for quick commit scanning.
`git log --graph --oneline --all`	Show visual ASCII commit history graph for all local and remote branches.
`git diff`	Show changes between working directory and staging area.
`git diff --staged`	Show changes between staged modifications and last commit.
`git diff <branch-a>..<branch-b>`	Compare code diffs directly between two specified branches.
`git blame <file>`	Display revision author and commit details line-by-line for a file.

9. UNDOING CHANGES & HISTORY REWRITING

COMMAND	DESCRIPTION
`git restore <file>`	Discard unstaged changes in working directory for a specific file.
`git restore --staged <file>`	Move file out of staging area back into working directory.
`git reset HEAD~1`	Undo latest commit; keep local modifications in working directory.
`git reset --hard HEAD~1`	Destructively discard latest commit and all uncommitted local modifications.
`git revert <commit-id>`	Create a new commit that safely reverses changes of a target commit.
`git rebase <branch>`	Re-apply current commits sequentially on top of another base tip.
`git rebase -i HEAD~N`	Start interactive rebase for last N commits (squash, reorder, edit, drop).

10. STASHING (TEMPORARY STORAGE)

COMMAND	DESCRIPTION
`git stash`	Temporarily stash all uncommitted local working directory changes.
`git stash save "message"`	Stash local modifications with a descriptive custom text label.
`git stash list`	List all saved stash stack items.
`git stash apply`	Re-apply most recent stash changes while keeping it on stash list.
`git stash pop`	Re-apply most recent stash changes and immediately remove it from list.
`git stash drop`	Delete the most recently created stash entry.
`git stash clear`	Permanently clear and remove all entries in the stash list.

11. GLOBAL CONFIGURATION

COMMAND	DESCRIPTION
`git config --global user.name "Name"`	Set global user name for commit attribution.
`git config --global user.email "email"`	Set global user email address.
`git config --global init.defaultBranch main`	Set default branch name automatically for all new initialized repositories.
`git config --list`	Display current combined global and local Git configurations.